

Software Architecture and Design Specification

Project: Smart Complaint Management System

Version: 1.0

Authors: Shashank SK- PES2UG24CS460 ,
Shreya Shenoy - PES2UG24CS487,
Sanjan M - PES2UG24CS442,
Samartha MS - PES2UG24CS435

Date: 02-10-2025

Status: Version 1 completed as of 03-10-2026

Revision History

Version	Date	Author	Change Summary
1.0	02-10-2026	Shreya Shenoy	Initial draft of the Software Architecture and Design Specification

Approvals

Role	Name	Signature	Date
Project Guide / Teacher	Sudeepa Ma'am		
Team Leader	Shasank S K	Shasank S K (digitally verified sign)	3-10-2026

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Smart Complaint Management System, including its major components, interfaces, data storage, security considerations, and design decisions.

1.2 Scope

The system covers user registration and login, complaint registration and categorization, complaint tracking and notifications, assignment, user-initiated escalation, status updates, smart complaint classification, duplicate complaint detection, administrator dashboard, reports and analytics, role-based access control, and search and filtering.

1.3 Audience

Developers, QA engineers, project evaluators, system administrators, and maintenance teams.

1.4 Definitions

SRS – Software Requirements Specification

SASD – Software Architecture and System Design

RBAC – Role-Based Access Control

UI – User Interface

Flask – Python web framework used for the system backend

ML - Machine Learning

2. Document Overview

2.1 How to use this document

This document presents the architecture and design of the Smart Complaint Management System, including its architectural overview, component design, architecture pattern, technology stack, risks and mitigations, requirements traceability, security architecture, API design, error handling, UX design, and future enhancements.

2.2 Related Documents

Software Requirements Specification (SRS), Software Test Plan (STP), and Requirements Traceability Matrix (RTM). (STP and RTM are included in the SRS document itself)

3. Architecture

3.1 Goals & Constraints

Goals:

- Provide a secure and reliable web-based complaint management system.

- Support the complete complaint lifecycle from registration and categorization through assignment, tracking, escalation, status updates, and resolution.
- Provide smart complaint classification and duplicate complaint detection to support complaint management.
- Ensure role-based access for Users and Administrators.
- Maintain data integrity, privacy, auditability, and fault tolerance throughout complaint processing.
- Provide a simple and consistent interface that can be used without specialized training.

Constraints:

- The system shall use Python, Flask, and MySQL as the defined technology stack.
- The system is a standalone academic prototype and shall not depend on external government, municipal, or third-party authentication systems.
- The system is intended for a limited deployment environment and is not designed for large-scale nationwide deployment.
- The system shall support commonly used modern desktop/laptop browsers, including Google Chrome, Mozilla Firefox, and Microsoft Edge.
- User personal and complaint-related information shall be protected from unauthorized access and unnecessary public exposure.
- Development is constrained by the stipulated academic timeline and available team resources.

3.2 Stakeholders & Concerns

Users: usability, security, privacy, reliability, availability

Administrators: maintainability, data integrity, auditability, efficient complaint management

Developers(Us- group 8) : modularity, maintainability, scalability

Project Evaluators: functionality, accuracy, reliability, compatibility

3.3 Component (UML) Diagram

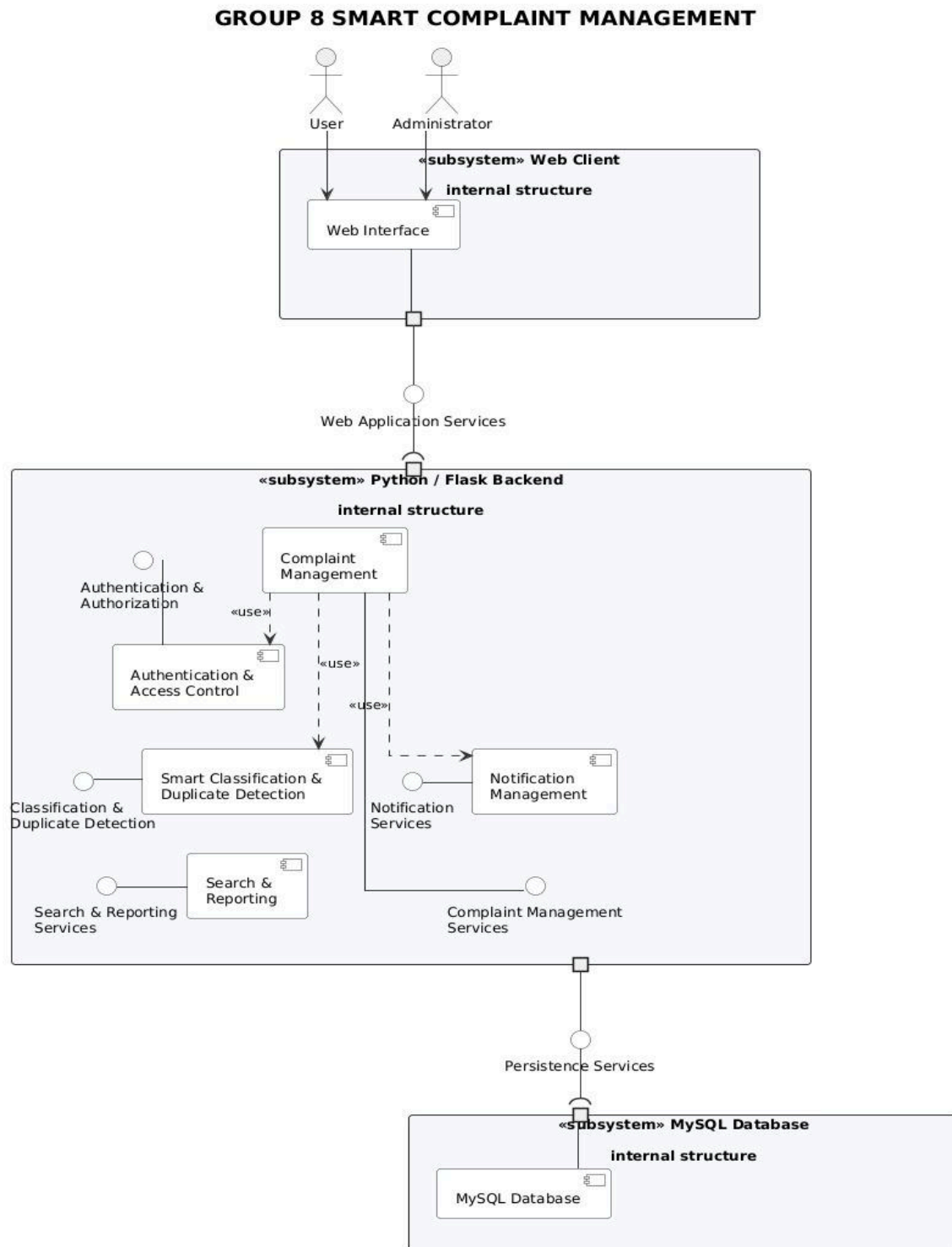


Figure: Component Diagram of the Smart Complaint Management System drawn on PlantUML

3.4 Component Descriptions

The Smart Complaint Management System is composed of three major subsystems: **Web Client, Python/Flask Backend, and MySQL Database**. The Python/Flask Backend contains the major logical components responsible for application functionality.

3.4.1 Web Interface

The Web Interface provides the user-facing interface through which the User and Administrator interact with the system. It allows users to access the functionality provided by the backend, including complaint-related operations, tracking, search, and administrative functions according to their assigned role.

The Web Interface communicates with the Python/Flask Backend through the Web Application Services interface. It does not directly access the MySQL database.

3.4.2 Authentication & Access Control

The Authentication & Access Control component manages user registration, login, authentication, and role-based access control.

It distinguishes between the two system roles, User and Administrator, and ensures that system functionality is accessed according to the permissions associated with each role. It provides the Authentication & Authorization interface to the other backend functionality.

3.4.3 Complaint Management

The Complaint Management component forms the core of the system's complaint-processing functionality. It handles complaint registration and the management of complaints throughout their lifecycle.

It supports functionality such as complaint categorization, assignment, escalation, status updates, and complaint tracking.

Complaint Management uses the Authentication & Access Control, Smart Classification & Duplicate Detection, and Notification Management components where required.

3.4.4 Smart Classification & Duplicate Detection

The Smart Classification & Duplicate Detection component provides the system's smart complaint-processing capabilities.

It supports priority categorization/classification of complaints and identifies complaints that may be duplicates of existing complaints. Potential duplicates are presented for administrative review; the component does not automatically merge, delete, or reject complaints.

3.4.5 Notification Management

The Notification Management component handles notifications associated with complaint-related events and updates.

It provides the Notification Services interface used by the Complaint Management component when notifications need to be generated.

The specific external delivery mechanism for notifications is not fixed by the system specification.

3.4.6 Search & Reporting

The Search & Reporting component provides functionality for searching and filtering complaint-related information and generating reports and analytics.

It supports the administrative analysis of complaint information and provides the Search & Reporting Services interface.

3.4.7 MySQL Database

The MySQL Database stores the persistent data required by the Smart Complaint Management System.

It acts as the system's data store and provides Persistence Services to the Python/Flask Backend.

The Web Interface does not directly communicate with the database; database access is handled through the backend.

3.5 Chosen Architecture Pattern and Rationale

Layered Architecture was selected as the primary architecture pattern because the Smart Complaint Management System naturally separates into a Presentation Layer (Web Interface), Application/Business Layer (Python/Flask Backend), and Data Layer (MySQL Database). This separation provides clear separation of concerns, maintainability, and controlled access to persistent data.

The following alternatives were considered too and rejected because:

- **Client-Server Architecture:** The system does exhibit a client-server interaction between the browser-based Web Interface and Flask Backend; however, Client-Server primarily describes the communication relationship and does not adequately represent the internal organization of the backend and data layers. Therefore, it is not used as the primary architecture pattern.
- **Pipe-and-Filter Architecture:** Not selected because the system does not primarily process data through a sequential chain of independent transformation stages.

Complaint management involves interactions among authentication, complaint processing, classification, notifications, search, and reporting rather than a fixed processing pipeline.

- **Object-Oriented Architecture:** Not selected as the primary architectural pattern because object-oriented programming is more closely related to the implementation/design paradigm of the software rather than the high-level organization of the Web Interface, backend, and database layers.
- **Event-Driven Architecture:** Not selected because the system's core operations are primarily request-response interactions initiated through the Web Interface rather than being organized around independent event producers and consumers.
- **Microservices Architecture:** Not selected because the project is a self-contained academic prototype with a single Flask backend and MySQL database. Separating the backend functionality into independently deployed services would introduce unnecessary deployment and communication complexity.
- **Black-Box Architecture:** The system components are treated as black boxes at their interfaces, with their internal implementation hidden. However, black-box design describes how components expose functionality through interfaces rather than providing the overall structural organization required for the system architecture; therefore, it is not selected as the primary architecture pattern.

Therefore, Layered Architecture is selected as the primary architecture pattern, while the system also exhibits client-server interaction and uses black-box component interfaces within its design.

3.6 Technology Stack & Data Stores

Python is used as the primary backend programming language, with Flask serving as the web application framework. The client is accessed through a modern desktop/laptop web browser, while MySQL is used as the relational database for persistent storage. The system uses standard web communication protocols, with HTTPS preferred for deployment to protect credentials and complaint-related information transmitted between the client and server.

The technology stack consists of:

- **Client:** Modern desktop/laptop web browser
- **Backend:** Python
- **Web Framework:** Flask
- **Database:** MySQL
- **Data Store:** MySQL relational database

The system does not specify a particular external notification provider, authentication service, or machine-learning framework. Therefore, these are not included as part of the defined technology stack.

3.7 Risks & Mitigations

Risk: Network failure may prevent users and administrators from accessing the web application → **Mitigation:** Require stable network connectivity and handle communication/application errors gracefully.

Risk: Unauthorized access to user or complaint information → **Mitigation:** Enforce authentication, role-based access control, and authorization checks.

Risk: Loss or accidental modification of complaint information → **Mitigation:** Validate important operations, preserve complaint records, and support appropriate backup and recovery procedures.

Risk: Incorrect priority classification or duplicate complaint detection → **Mitigation:** Maintain sufficient accuracy during testing and allow administrators to review and update classification results where applicable.

Risk: Notification mechanism may become unavailable → **Mitigation:** Use the configured notification mechanism and ensure the system handles notification-related failures appropriately.

Risk: Invalid or incomplete user input may affect system operations → **Mitigation:** Perform input validation and display appropriate error messages before accepting data.

3.8 Traceability to Requirements

SRS Requirement	Requirement / Feature	Responsible Component(s)
REQ-1.1 – REQ-1.7	User Registration and Login	Authentication & Access Control
REQ-2.1 – REQ-2.6	Complaint Registration	Complaint Management
REQ-3.1 – REQ-3.5	Complaint Categorization	Complaint Management

REQ-4.1 – REQ-4.5	Complaint Tracking and Notifications	Complaint Management, Notification Management
REQ-5.1 – REQ-5.5	Complaint Assignment	Complaint Management
REQ-6.1 – REQ-6.5	Complaint Escalation	Complaint Management
REQ-7.1 – REQ-7.5	Smart Complaint Classification (Priority Categorization)	Smart Classification & Duplicate Detection
REQ-8.1 – REQ-8.5	Duplicate Complaint Detection	Smart Classification & Duplicate Detection
REQ-9.1 – REQ-9.6	Complaint Status Updates	Complaint Management
REQ-10.1 – REQ-10.6	Admin Dashboard	Web Interface, Complaint Management
REQ-11.1 – REQ-11.5	Reports and Analytics	Search & Reporting
REQ-12.1 – REQ-12.6	Role-Based Access Control	Authentication & Access Control
REQ-13.1 – REQ-13.6	Search and Filtering	Search & Reporting

3.9 Security Architecture

Threat Modeling (using STRIDE):

Spoofing → User authentication

Tampering → Input validation and audit records

Information Disclosure → HTTPS and access restrictions

Denial of Service → Availability and fault-tolerance measures

Elevation of Privilege → Role-Based Access Control (RBAC) and authorization checks

4. Design

4.1 Design Overview

The Smart Complaint Management System is designed using a **layered architecture** that separates the Web Interface, Python/Flask Backend, and MySQL Database into distinct layers. This separation of concerns supports **maintainability, modularity, data integrity, and controlled access to persistent data**.

4.2 UML Sequence Diagrams

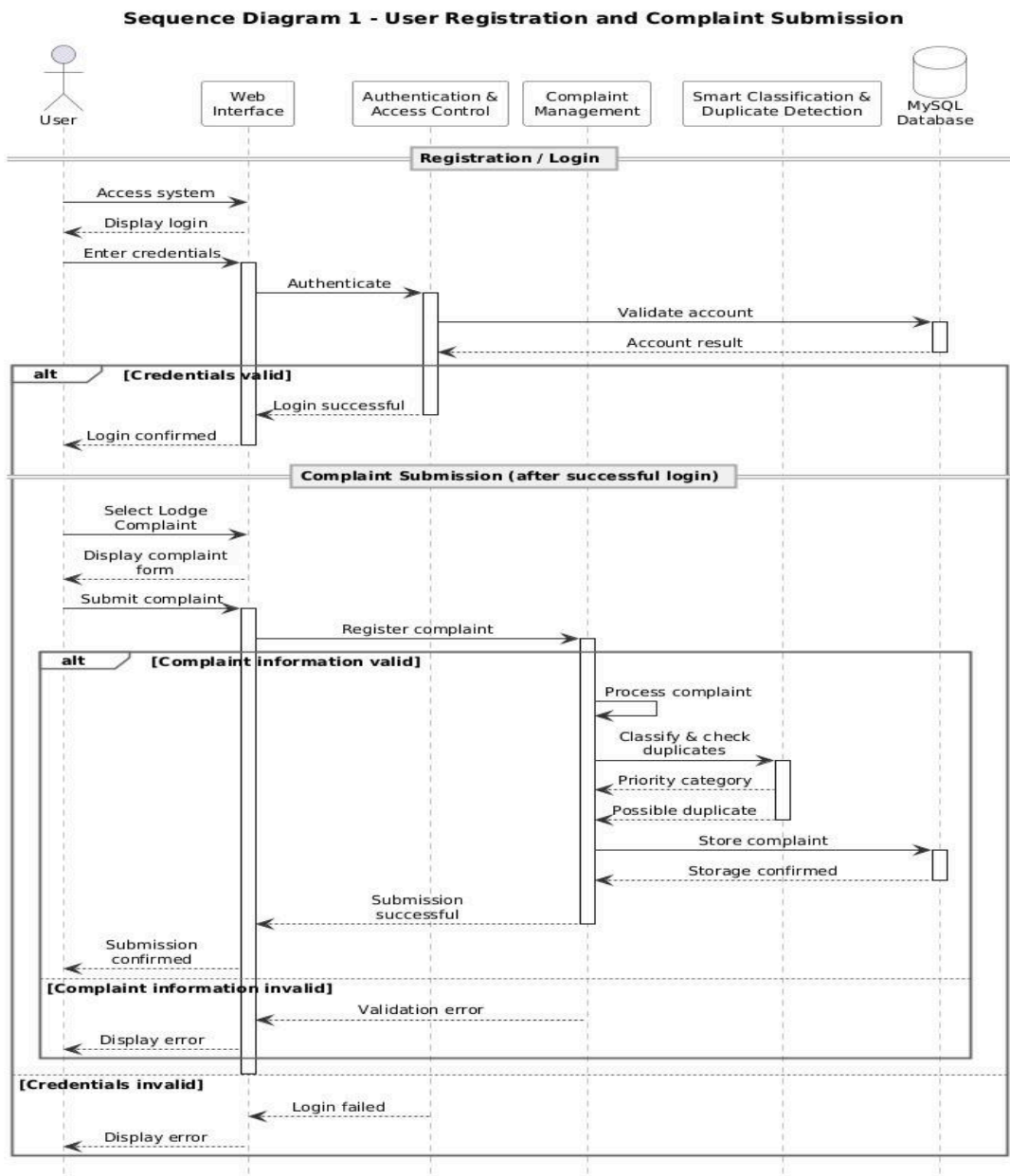


Figure 1: Sequence of interactions for user authentication, complaint submission, smart classification and duplicate detection, and storage of complaint information. (generated on plantUML)

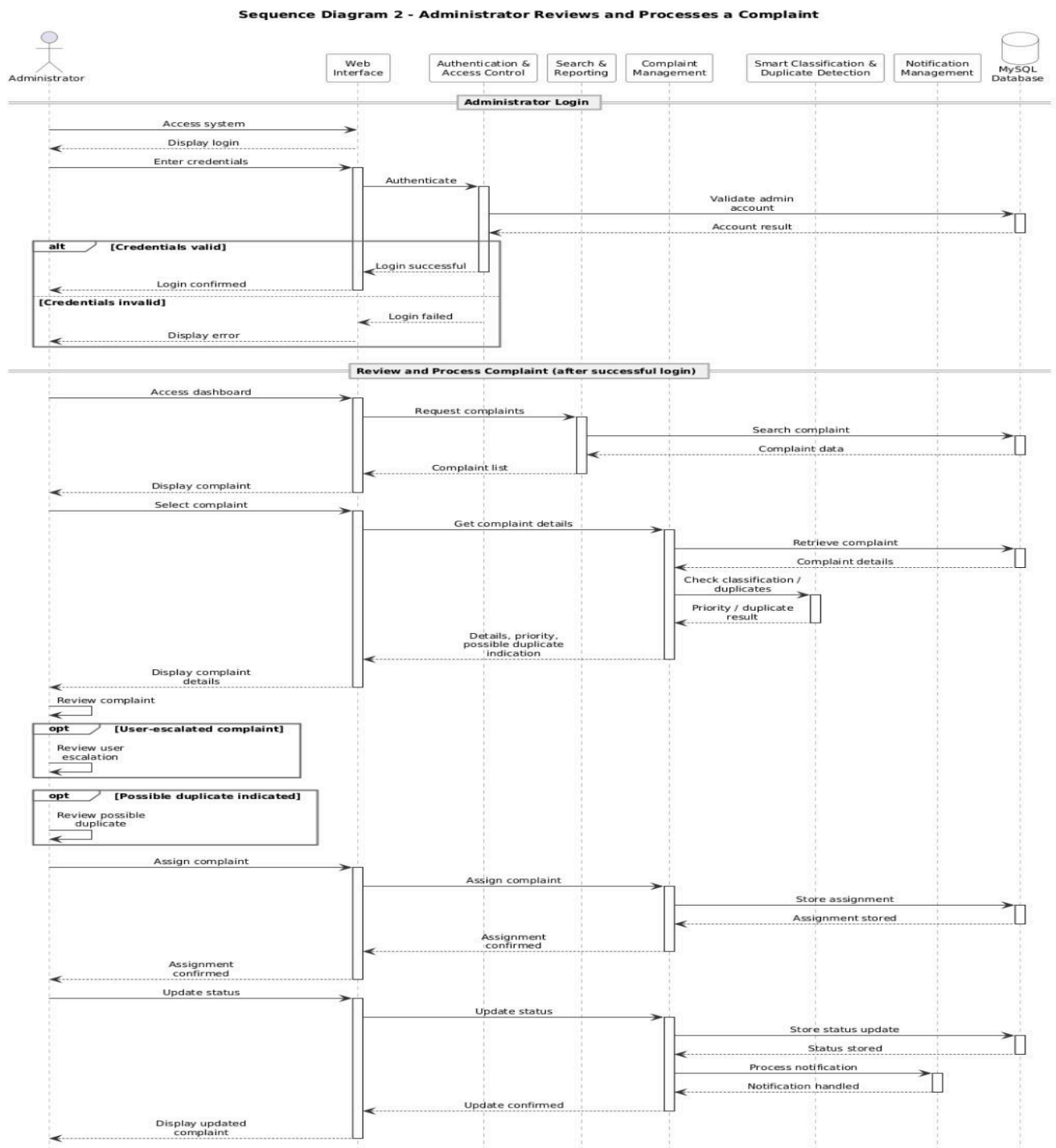


Figure 2: Sequence of interactions for administrator authentication, complaint review, assignment, status update, notification handling, and database processing. (generated on plantUML)

4.3 API Design

The system exposes interfaces between the Web Interface and the Python/Flask Backend components. The following interfaces define the operations supported by major backend components. Specific implementation-level endpoint paths and data formats may be finalized during development.

4.3.1 Authentication & Access Control Interface

Operation: User/Administrator Authentication

Method: POST

Request: User authentication credentials

Response: Authentication result and authorized role information

Errors: Invalid credentials, unauthorized access

Purpose: Authenticates the User or Administrator and determines access according to the assigned role.

4.3.2 Complaint Management Interface

Operation: Complaint Registration

Method: POST

Request: Complaint information submitted by the User

Response: Complaint submission result and complaint information

Errors: Invalid or incomplete complaint information, unauthorized access

Purpose: Registers and processes complaints submitted by Users and supports subsequent complaint lifecycle operations such as assignment, tracking, escalation, and status updates.

4.3.3 Smart Classification & Duplicate Detection Interface

Operation: Complaint Classification and Duplicate Detection

Method: POST

Request: Complaint information

Response: Priority categorization and possible-duplicate indication

Errors: Invalid complaint information, processing failure

Purpose: Performs the system's smart complaint-processing functions and provides the resulting priority classification and possible duplicate indication to the Complaint Management component

Complete API design table with all components listed briefly in tabular form:

Component	Operation	Method	Purpose
Authentication & Access Control	Authenticate	POST	Send credentials securely
Complaint Management	Register complaint	POST	Create a new complaint
Search & Reporting	Retrieve/search complaints	GET	Retrieve/filter existing complaint data
Complaint Management	Update complaint status	PUT/PATCH	Modify existing complaint
Complaint Management	Assign complaint	PUT/PATCH	Update assignment
Smart Classification & Duplicate Detection	Classify/check duplicates	POST	Send complaint content for processing

4.4 Error Handling, Logging & Monitoring

The system shall provide standardized and user-friendly error messages for invalid inputs, authentication failures, unauthorized access, and other processing errors. Error messages displayed to users shall not expose sensitive information such as passwords or protected complaint data.

System activities and significant complaint-processing events shall be recorded through audit records, particularly actions such as complaint assignment, status updates, and escalation. Logging shall avoid storing sensitive information unnecessarily.

Monitoring shall focus on the availability, reliability, and operational correctness of the system. Relevant indicators include failed transactions or requests, authentication/access failures, complaint-processing failures, and system availability during testing and deployment. The system shall support appropriate error handling and recovery to maintain reliable operation.

4.5 UX Design

The system shall provide a simple and consistent web interface for both Users and Administrators. The interface shall support clear navigation, readable text, appropriate input validation, and understandable error messages. Access to functions and information shall be presented according to the user's role. The design shall support accessibility and usability so that users can interact with the system without specialized training.

4.6 Open Issues & Next Steps

Future enhancements: Improved smart classification and duplicate detection accuracy using better ML models, enhanced reporting and analytics, expanded scalability for larger deployments, and additional accessibility improvements.

5. Appendices

5.1 Glossary:

SRS – Software Requirements Specification

SASD – Software Architecture and System Design

RBAC – Role-Based Access Control

UI – User Interface

API – Application Programming Interface

HTTPS – Hypertext Transfer Protocol Secure

ML – Machine Learning

STRIDE – Security threat-modeling framework covering Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege

5.2 References:

Smart Complaint Management System – Software Requirements Specification (SRS), Version 1.0

Software Test Plan (STP) for the Smart Complaint Management System

Requirements Traceability Matrix (RTM) for the Smart Complaint Management System

5.3 Tools:

PlantUML to generate component and sequence diagram,
Google docs to prepare the document